Antihydra

From raw tape to one conjecture — a technical walkthrough

`1RB1RA_0LC1LE_1LD1LC_1LA0LB_1LF1RE_---0RA`

AH

Core Lean 4 · no Mathlib · axioms [propext, Quot.sound] · no sorry

The machine

	read 0	read 1
A	1RB	1RA
B	0LC	1LE
C	1LD	1LC
D	1LA	0LB
E	1LF	1RE
F	— halt —	0RA

- Six states, two symbols, one undefined transition: F reading 0 is the only way to halt
- Tape = total function $\mathbb{Z} \to$ Bool; a configuration is (state, head, tape)
- The table is hand-transcribed once — then kernel-proved equal to the parsed standard-format code (parse_ok, with corrupted-code negative controls)

Halt localization

1

step = none

⇔ state F reads 0

`step_eq_none_iff`

2

F entered only via E0

write 1, move left

`into_F_only_from_E0`

3

any halt ⇒ E-probe of 00

a reachable E reading 0 with 0 to its left

`halt_requires_e_probe_00`

4

probe-safety ⇒ never halts

every probed 0 has a 1 on its left

`nonhalting_of_e_probe_safe`

Three rules, exact for all block values

HYDRA $m \geq 3$

$(a, m, c) \to (a, m-2, c+3)$

$2c + 11$ raw steps

counter frozen — ~96% of all dynamics

DROP $m = 2$

$(a, 2, c) \to (a-1, c+3, 2)$

$2c + 15$ raw steps

counter pays 1 — dies if $a \leq 1$

MERGE $m = 1$

$(a, 1, c) \to (a+2, c+2, 2)$

$2a + 2c + 22$ raw steps

counter gains 2

Each rule is a theorem: the exact raw trajectory plus an ESafeSeg certificate — every E-probe of a 0 during the rule provably has a 1 to its left. The proofs enumerate the probes one by one.

The abstract orbit and its conjecture

- aorbit: iterate the three rules from (2, 7, 2) — reached from blank tape by a proven 62-step startup
- Death set (exact): $m = 0 \lor (m = 2 \land a \leq 1)$ — astep_dies_iff
- SafeOrbitA : the orbit never dies
- Transport lemmas realize every abstract state as a tape pattern; segment induction stitches safe segments into probe safety

MAIN THEOREM

```
safeOrbitA_nonhalting :
  SafeOrbitA →
  ¬ RawHalts blankStart
```

```
axioms: [propext, Quot.sound]
```

Everything about the machine is proven. The single assumption is a statement about the integer orbit alone — and the next slide shows it is the community's own conjecture.

The conjugacy: orbit ⇔ hydra process

hydra value	`b = m + 2·(c−2)/3 + 5`
heads counter	`= a  (the counter block)`
even b step	`= merge (heads + 2)`
odd b step	`= drop (heads − 1)`
death	`odd b with heads ≤ 1`
update	`b ← b + ⌊b/2⌋`

`relHA_step: the two systems run in lock-step; deaths coincide exactly.`
`hydraSafe_iff_safeOrbitA · hydraSafe_nonhalting`

Honest delta: formal death at heads ≤ 1 vs the wiki's 0 — one boundary case stronger; open follow-up.

Incision events and the closed-form volley

$$\texttt{volley}\ (a, m, c) \;=\; (a,\ \ 2 - m \bmod 2,\ \ c + 3\cdot\lfloor (m-1)/2 \rfloor)$$

one formula replaces the whole hydra run

- estep = boundary rule + volley: exactly one event per counter movement
- eventSafe_iff_safeOrbitA: the sampled process dies ⇔ the full orbit dies
- Soundness AND completeness — checking between events is provably redundant
- eventSafe_nonhalting: event safety alone already implies the machine never halts

WHY IT PAYS

volleys grow without bound: the k-th event skips ~b^2 raw steps, b ≈ 12·1.5^k

event 16 ≈ step 10^8
event 20,000 ≈ step 10^{7046}

Certified acceleration, measured



10^8

raw steps covered by the first 16 events

7,046

digits in the raw-step count matched by 20k events

Axiom hygiene: keeping Classical.choice out

CONTAMINATES

- omega proving a positive ∧ / ∨ goal
- decide / rfl through String.toList
- quantified decide over finite inductives (fails outright in core)

STAYS CLEAN

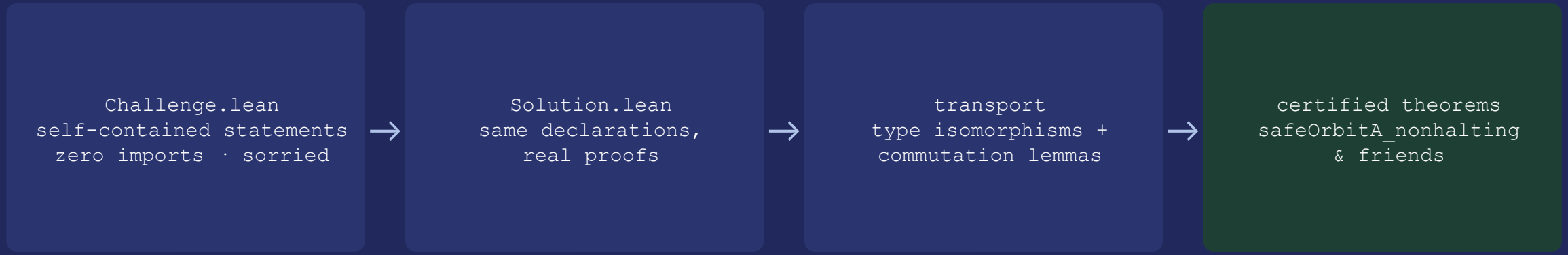
- omega on atomic goals — split with constructor, congr 1, Or.inl
- decide over List Char literals
- cases a <;> cases b <;> simp_all for finite injectivity

A gate script rebuilds all packages and fails on any axiom outside [propext, Quot.sound] — the two quarantined String display lemmas are the only documented exception.

Is it the right machine? Parity evidence

`parse_ok`	in-Lean parser of the standard format, kernel-proved equal to the hand-written table; corrupted-code controls prove it non-vacuous	`axioms: [propext]`
`trace ⇔ reference`	the proved step function, run executably, is byte-identical for 2,000 steps vs an independent 30-line Python stepper (both machines)	`4/4 tests green`
`startup anchors`	the proven milestones appear on schedule in the independent trace: step 62 → B@6 (Antihydra), step 35 → E@5 (dossier-1)	`matches exactly`

Palomar packaging: Challenge / Solution / transport



- Comparator (Lean kernel + independent NanoDa kernel) will confirm Solution proves exactly the Challenge, using only permitted axioms
- All three entries pass the full local intake contract; challenges are 79–153 lines, comfortably auditable
- All four compared theorems: [propext, Quot.sound] — strictly inside Palomar's allowed set
- Remaining before submission: LICENSE choice, git push, owner sign-off

One machine, one conjecture

HydraSafe → Antihydra never halts

- The machine's entire dynamics are theorems; the open part is pure arithmetic
- Boundary-only checking is certified — the conjecture can be hunted at 10^{7046}-step scale
- The same architecture is ready for the next cryptid

bbchallenge-proofs · docs/learning · docs/results/2026-08-22-*